셰이더 프로그래밍: OpenGL 렌더링 구조와 데이터 준비

Lecture 2 | 한국공학대학교 게임공학과 | 이택희 교수

한국공학대학교
TECH UNIVERSITY OF KOREA

CPU와 GPU의 평행 우주: 완벽한 분업

CPU가 게임 로직을 처리하는 동안, GPU는 독립적으로 화면을 그립니다.
서로 기다리지 않는 극대화된 병렬 처리(Parallelism)가 핵심입니다.

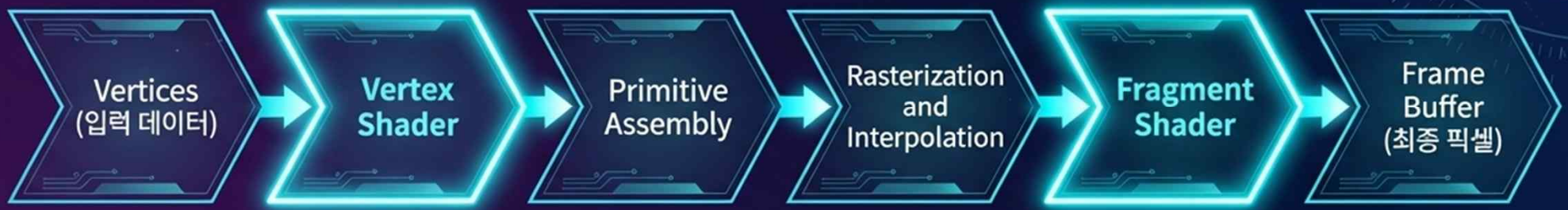
한국공학대학교
TECH UNIVERSITY OF KOREA

어떻게 명령을 내릴 것인가? OpenGL 상태 머신 (State Machine)

OpenGL은 일반적인 C++ 객체처럼 작동하지 않습니다.
거대한 하나의 '상태 머신'입니다.

CPU는 매 프레임마다 모든 데이터를 새로 넘기는 것이 아니라, 상태(State)를 설정(Bind)하고 그리기(Draw) 명령을 내릴 뿐입니다. 설정된 상태는 명시적으로 바꾸기 전까지 유지됩니다.

그래픽스 렌더링 파이프라인 (The Graphics Pipeline)

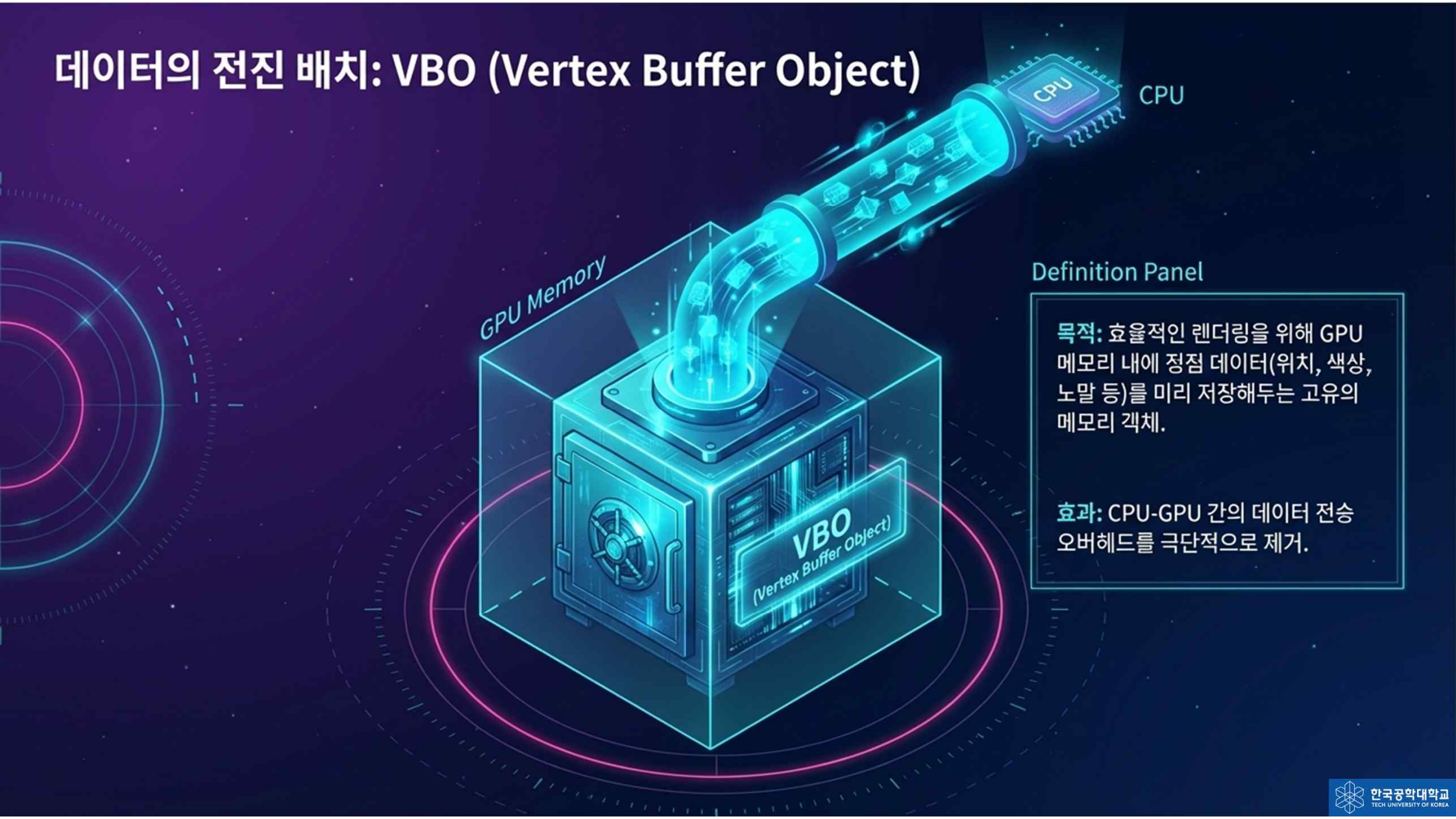


핵심 포인트: 이 거대한 파이프라인을 가동하기 위한 첫 번째 단계는 바로 'Vertex Shader'에 'Vertices(정점 데이터)'를 입력하는 것입니다.

파이프라인의 첫 번째 허들: 메모리의 분리

효율적인 렌더링을 위해서는 CPU가 GPU에게 데이터를 매번 보내는 것이 아니라,
GPU 내부에 고유의 Data 형식을 생성하고 미리 넘겨두어야 합니다.

데이터의 전진 배치: VBO (Vertex Buffer Object)

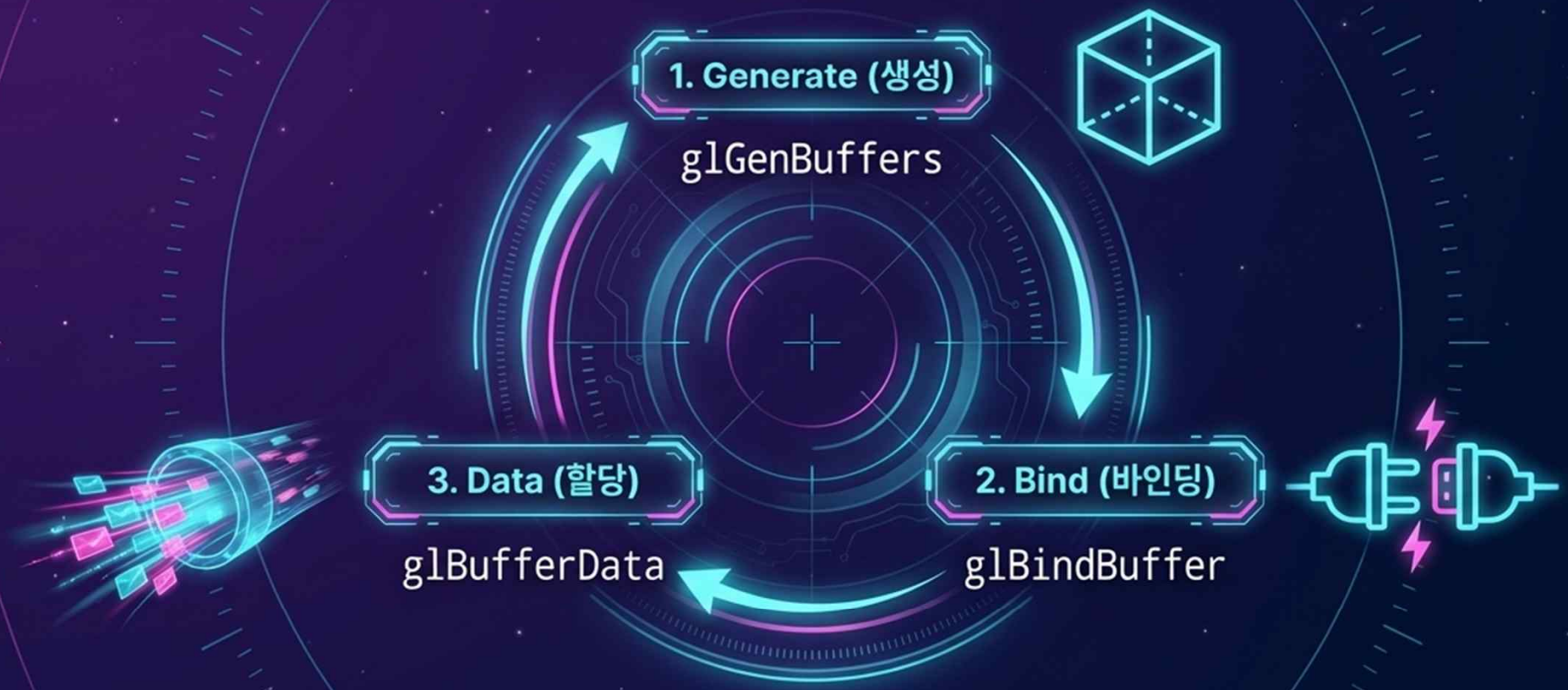


Definition Panel

목적: 효율적인 렌더링을 위해 GPU 메모리 내에 정점 데이터(위치, 색상, 노말 등)를 미리 저장해두는 고유의 메모리 객체.

효과: CPU-GPU 간의 데이터 전송 오버헤드를 극단적으로 제거.

OpenGL 객체 제어의 핵심 패턴: Generate & Bind



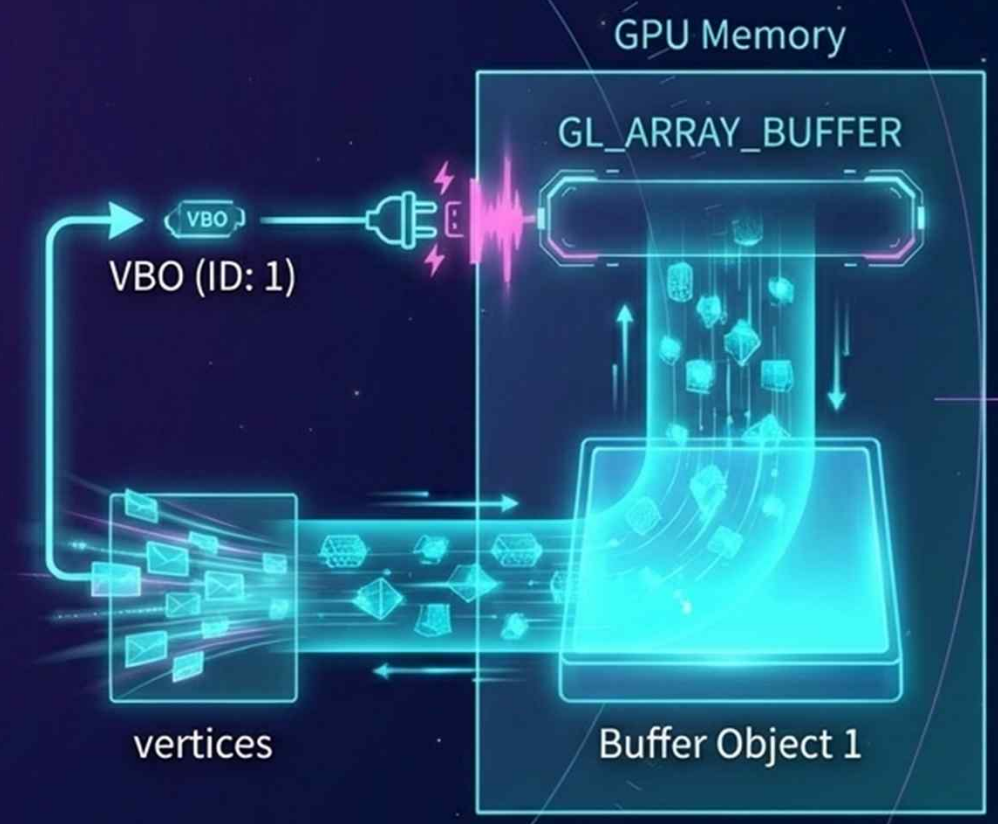
OpenGL에서는 직접 객체를 조작하지 않습니다.
ID를 발급받고, 타겟에 바인딩한 후, 타겟을 통해 데이터를 조작합니다.

한국공학대학교
TECH UNIVERSITY OF KOREA

실전: 코드가 물리적 메모리를 움직이는 과정

```
float vertices[] = {0.0f, 0.0f, 0.0f,
                    1.0f, 0.0f, 0.0f, ...};
GLuint VBO;

glGenBuffers(1, &VBO);
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER,
             sizeof(vertices), vertices,
             GL_STATIC_DRAW);
```



한국공학대학교
TECH UNIVERSITY OF KOREA

데이터 준비 완료 (Summary & Next Steps)

System Status: GREEN

- [✓] CPU-GPU 평행 아키텍처 이해 (참견 X)
- [✓] 상태 머신(State Machine) 패러다임 적응
- [✓] VBO 생성 및 GPU로 데이터 전송 완료 (Generate -> Bind -> Data)

Next: Vertex Shader

이제 GPU에 안전하게 저장된 이 데이터를 어떻게 요리할 것인가?
다음 시간, 정점 셰이더 프로그래밍에서 이어집니다.